Test Plan for
RANDOM ACCESS MARKET

ChangeLog

Version	Change Date	By	Description
version number	Date of Change	Name of person who made changes	Description of the changes made
1.0	4/5/2026	Jackie	
1.1	4/7/2026	Jackie	
2.0	5/1/2026	Jackie	

1 INTRODUCTION 2

1.1 SCOPE 2

1.1.1 In-Scope 2

1.1.2 Out-of-Scope 2

1.2 QUALITY OBJECTIVE 2

1.3 ROLES AND RESPONSIBILITIES 2

2 TEST METHODOLOGY 3

2.1 OVERVIEW 3

2.2 TEST LEVELS 3

2.3 BUG TRIAGE 3

2.4 SUSPENSION CRITERIA AND RESUMPTION REQUIREMENTS 4

2.5 TEST COMPLETENESS 4

3 TEST DELIVERABLES 4

4 RESOURCE & ENVIRONMENT NEEDS 4

4.1 TESTING TOOLS 4
4.2 TEST ENVIRONMENT 5

5 TERMS/ACRONYMS 5

1 Introduction

Brief introduction of the test strategies, process, workflow and methodologies used for the project

1.1 Scope

1.1.1 In-Scope

- Public page routing
- Authentication behavior for valid and invalid access attempts

- Role-based authorization for buyer, seller, and administrator routes
- Buyer API behavior for product browsing and cart
- Seller API behavior for inventory viewing and product creation
- Database-backed behavior using Prisma and MySQL, including user retrieval and login token persistence
- Seller wallet and mock bank payout behavior
- Buyer order history, mock tracking, and order detail behavior
- Buyer return submission and seller return review behavior

1.1.2 Out-of-Scope

- Actual payment processor integration

1.2 Quality Objective

- Ensure the Application Under Test conforms to functional and non-functional requirements
- Ensure the AUT meets the quality specifications defined by the client
- Bugs/issues are identified and fixed before go live
- Ensure core buyer, seller, and administrator behaviors are validated through automated tests

1.3 Roles and Responsibilities

- QA Analyst
- Test Manager
- Configuration Manager
- Developers
- Installation Team

Name	Net ID	GitHub username	Role
Jackie Cline	jjc446	i-like-burgers	
Daniel Beauker	djb675	Daniel-0-bit	

2 Test Methodology

2.1 Overview

Ideally, we are aiming for (an abomination of) agile as our test methodology.

2.2 Test Levels

The system is built around a Svelte + Vite frontend, ExpressJS routes and middleware, Prisma, and MySQL, so integration tests that verify components working together are of high value here. Tests currently use Jest and Supertest to exercise route behavior, authentication, role-based authorization, buyer flows, seller flows, and admin flows through HTTP requests.

Database-integration tests are incorporated to verify that Express routes correctly interact with Prisma and MySQL. Such testing includes registration, login, token persistence and retrieval of database-backed records. There are also additional tests of this type that utilize mocked authentication or mocked dependencies so route behavior can be tested without demanding full database setup for every case.

Additionally to local execution, the project also incorporates a Github Actions CI workflow tht runs on push and pull request events for the configured staging and main branches. The current workflow installs dependencies, generates the Prisma client, applies the schema with 'prisma db push', seeds the database, runs the Jest suite, generates a JSON Jest report, and uploads that report as an artifact.

2.3 Bug Triage

Bugs discovered during testing are reviewed based on severity, affected functionality, and impact on core project requirements. Issues affecting authentication, access control, cart behavior, inventory behavior, or database integrity are treated as higher priority because they affect core user stories and overall system correctness.

Priority Order:

- High: login failures, broken route protection, broken database writes, cart/inventory corruption
- Medium: incorrect validation behavior, moderate UI/ and route issues, incomplete non-critical workflows
- Low: pure cosmetic issues, non-blocking wording or formatting issues, incomplete polish

Open defects and functional gaps:
Some user stories are not currently implemented, not fully functional, or are not yet validated by automated testing. These entries do not constitute passed behavior. Rather than omitting them entirely however, these are explicitly documented.

- Functional gap: story exists, but implementation is incomplete or absent
- Testing gap: implementation may exist, but automated or manual verification incomplete

DEF-40
User Story: #40 Updating Buyer Display Name
Status: Functional Gap
Summary: Buyer display name update behavior is not currently covered as a stable implemented feature.

~~DEF-39~~
~~User Story: #39 Seller bank information~~
~~Status: Functional Gap~~
~~Summary: Seller bank information linkage is not currently validated and appears incomplete or absent.~~

DEF-37
User Story: #37 Saving Payment Information to Buyer Account
Status: Functional Gap
Summary: Saved payment information behavior is not currently validated and appears incomplete or absent.

DEF-34
User Story: #34 Remove Product from Inventory
Status: Functional Gap
Summary: Seller inventory removal behavior is not currently validated and does not have confirmed stable coverage.

~~DEF-26~~
~~User Story: #26 Viewing past purchases~~
~~Status: Functional Gap~~
~~Summary: Past purchase history behavior is not currently validated and appears incomplete or absent.~~

DEF-25
User Story: #25 Updating Seller Contact Information
Status: Functional Gap
Summary: Seller contact information update behavior is not currently validated and appears incomplete or absent.

DEF-23
User Story: #23 Changing Buyer Profile Picture
Status: Functional Gap
Summary: Buyer profile picture update behavior is not currently validated and appears incomplete or absent.

DEF-22
User Story: #22 Updating Buyer Shipping Address
Status: Functional Gap
Summary: Buyer shipping address update behavior is not currently validated and appears incomplete or absent.

DEF-21
User Story: #21 Updating seller profile description
Status: Functional Gap
Summary: Seller description update behavior is not currently validated and appears incomplete or absent.

DEF-20
User Story: #20 Updating seller profile image
Status: Functional Gap
Summary: Seller profile image update behavior is not currently validated and appears incomplete or absent.

DEF-19
User Story: #19 Update Seller Profile Name
Status: Functional Gap
Summary: Seller profile name update behavior is not currently validated and appears incomplete or absent.

DEF-18
User Story: #18 View and filter accounts by banned or active status
Status: Testing Gap
Summary: Admin user listing is covered, but explicit banned/active filtering behavior is not yet fully validated.

~~DEF-27~~
~~User Story: #27 Product Search~~
~~Status: Testing Gap~~
~~Summary: Product listing retrieval is covered, but explicit search-by-name behavior is not yet fully validated.~~

2.4 Suspension Criteria and Resumption Requirements

Suspension:

- The application cannot successfully start
- The database connection is unavailable or unstable
- Critical dependency failure prevents meaningful testing
- Major schema or backend changes render tests momentarily invalid until updates are applied

Resumption:

- The application successfully starts again
- Database and necessary environment configuration are available
- Critical dependency failure that prevents meaningful testing is resolved
- Outdated tests are updated to match current implementation

2.5 Test Completeness

Here you define the criterias that will deem your testing complete.
For instance, a few criteria to check Test Completeness would be

- 100% test coverage
- All Manual & Automated Test cases executed
- All open bugs are fixed or will be fixed in next release
- Core implemented flows have coverage in the current suite, especially login, protected routes, buyer product access, cart behavior, seller inventory access, and seller product creation

3 Test Deliverables

Here are the deliverables

- Test Plan
- CI completion CSV mapping use cases to implemented tests
- Saved Jest test report artifact
- Requirements Traceability Matrix

4 Resource & Environment Needs

4.1 Testing Tools

- Jest

- Supertest
- Github Actions

4.2 Test Environment

It mentions the minimum **hardware** requirements that will be used to test the Application.

- 64-bit Ubuntu Linux (or Windows)
- Node.js 24
- Npm
- MySQL database access
- Environment must support Prisma client generation

5 Terms/Acronyms

Make a mention of any terms or acronyms used in the project

TERM/ACRONYM	DEFINITION
API	Application Program Interface
AUT	Application Under Test